

JAVA DEVELOPMENT

CODANDO O PRIMEIRO CÓDIGO JAVA



2

LISTA DE FIGURAS

Figura 1 – Site do IntelliJ IDEA	6
Figura 2 – Download do IntelliJ IDEA Community	7
Figura 3 – Arquivo de instalação do IntelliJ IDEA.....	7
Figura 4 – Processo de instalação do IntelliJ IDEA (1)	8
Figura 5 – Processo de instalação do IntelliJ IDEA (2)	8
Figura 6 – Processo de instalação do IntelliJ IDEA (3)	9
Figura 7 – Processo de instalação do IntelliJ IDEA (4)	9
Figura 8 – Processo de instalação do IntelliJ IDEA (5)	10
Figura 9 – Processo de instalação do IntelliJ IDEA (6)	10
Figura 10 – Termos de uso do IntelliJ IDEA Community	11
Figura 11 – Criando um projeto Java no IntelliJ (1).....	11
Figura 12 – Criando um projeto Java no IntelliJ (2).....	12
Figura 13 – Criando um projeto Java no IntelliJ (3).....	12
Figura 14 – Criando um projeto Java no IntelliJ (4).....	13
Figura 15 – Criando um projeto Java no IntelliJ (5).....	13
Figura 16 – Execução do primeiro programa Java no IntelliJ.....	14
Figura 17 – Abrindo a classe Java compilado no Explorer.....	15
Figura 18 – Verificando o bytecode gerado.....	15
Figura 19 – Criando um pacote (1).....	17
Figura 20 – Criando um pacote (2).....	18
Figura 21 – Criando um pacote (3).....	18
Figura 22 – Abrindo o pacote no Explorer do Windows	19
Figura 23 – Projeto no Explorer do Windows	19
Figura 24 – Criando a classe Java (1).....	20
Figura 25 – Criando a classe Java (2).....	20
Figura 26 – Classe Produto.....	21
Figura 27 – Classe Main	22
Figura 28 – Exemplo de tipos primitivos.....	26
Figura 29 – Executando a classe	27
Figura 30 – Exercício 01	30
Figura 31 – Operadores de incremento e decremento	31
Figura 32 – Exemplo da utilização dos operadores.....	32
Figura 33 – Resultado da execução	32

LISTA DE TABELAS

Tabela 1 – Palavras reservadas (1)	24
Tabela 2 – Palavras reservadas (2)	24
Tabela 3 – Operadores aritméticos	28
Tabela 4 – Operadores de atribuição	29

EXEMPLO

SUMÁRIO

CODANDO O PRIMEIRO CÓDIGO	5
1 FERRAMENTAS DE DESENVOLVIMENTO.....	5
2 TIPOS DE DADOS	22
2.1 Manipulando dados	26
2.1.1 Operadores aritméticos	27
2.1.2 Operadores de atribuição	29
2.1.3 Operadores de incremento e decremento	30
REFERÊNCIAS	33

CODANDO O PRIMEIRO CÓDIGO

1 FERRAMENTAS DE DESENVOLVIMENTO

Vimos, no primeiro capítulo, o que é a plataforma Java, suas características, as edições e as várias siglas que envolvem esse novo mundo no qual estamos entrando! Fizemos o nosso primeiro programinha Java para entender o processo de compilação e interpretação do bytecode na JVM. Você deve ter percebido que é possível criar um código Java com bloco de notas, ou seja, é possível “codar” em qualquer editor de texto que possa gravar um arquivo com a extensão “.java”. Mas, como você é esperto, você deve imaginar que existem ferramentas mais apropriadas para os desenvolvedores. E você está certo!

Para desenvolver as aplicações, existem as ferramentas específicas, chamadas de IDE (*Integrated Development Environment* ou ambiente de desenvolvimento integrado). Essas ferramentas têm o objetivo de aumentar a produtividade e a eficiência do desenvolvedor. Existem IDEs para linguagens específicas e outras que suportam uma variedade de linguagens de programação. Todas essas ferramentas possuem funcionalidades similares, como as mais comuns:

- **Editor:** para escrever o código-fonte na linguagem de programação suportada pela IDE.
- **Compilador:** para compilar o código-fonte.
- **Depurador (*Debugger*):** para executar o programa “passo a passo”, ou seja, é possível verificar o que está acontecendo em cada instrução do programa, facilitando o entendimento do sistema e no processo de encontrar e corrigir bugs.
- **Geração de código:** para gerar código-fonte a partir de *templates* de código comumente utilizados para solucionar problemas rotineiros.
- **Distribuição (*Deploy*):** atua no auxílio do processo de gerar o arquivo final para a instalação do sistema desenvolvido ou até mesmo envia os arquivos para o servidor.

Codando o primeiro código Java

- **Git:** integração com o versionador de arquivos mais utilizados pelos desenvolvedores.
- **Modelagem:** para criação de modelos de classes, objetos, interfaces, associações e interações de forma visual.
- **Testes automatizados:** para realizar testes no programa de forma automatizada, baseados em scripts ou programas de testes previamente especificados, gerando relatórios que auxiliam na análise do impacto das alterações do código-fonte.
- **Refatoração:** realiza a melhoria constante do código-fonte, pode ser na construção de código mais otimizado, limpo e/ou com melhor entendimento pelos envolvidos no desenvolvimento do sistema.

Para desenvolver sistemas com Java e outras linguagens que também são interpretadas na JVM, existem várias opções no mercado. As mais utilizadas são Eclipse, IntelliJ, NetBeans, VSCode, JDeveloper.

Iremos optar pelo IntelliJ, uma poderosa IDE muito utilizada pelos desenvolvedores de várias linguagens. Existem duas versões, uma paga e outra gratuita, com menos recursos. Vamos utilizar a versão Community, que é grátis e nos atenderá muito bem. Você pode utilizar a ferramenta de sua preferência.

Para realizar a instalação do IntelliJ, navegue até o endereço: <https://www.jetbrains.com/pt-br/idea/> e clique no botão “Baixar”.

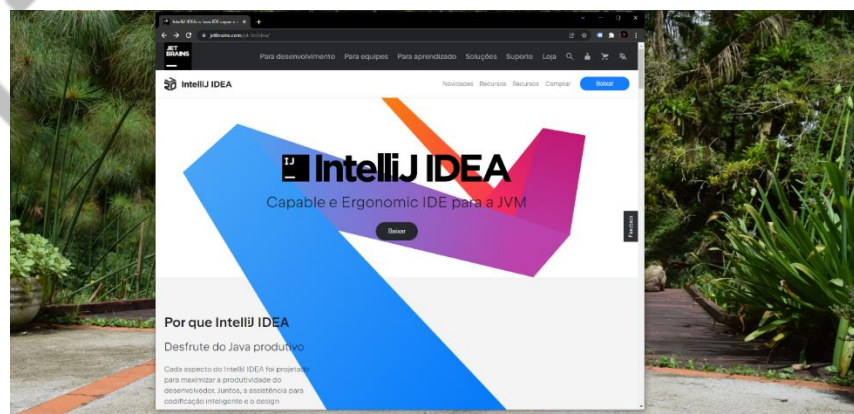


Figura 1 – Site do IntelliJ IDEA
Fonte: Elaborado pelo autor (2022)

Codando o primeiro código Java

Escolha o sistema operacional adequado e faça o download da versão “Community”.

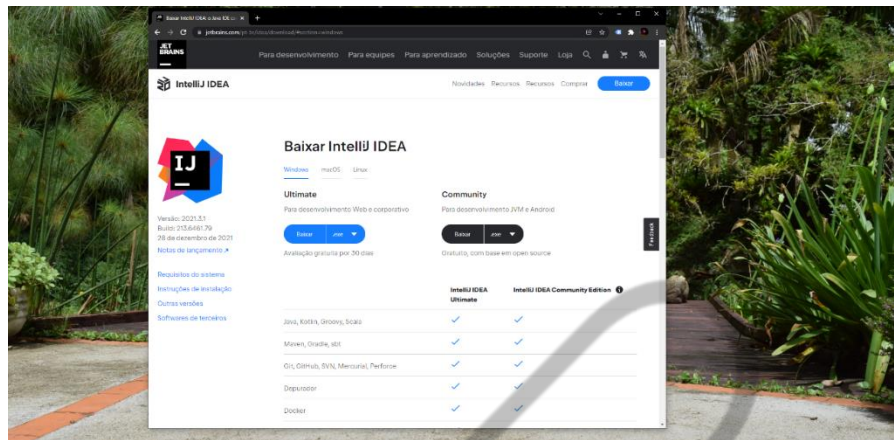


Figura 2 – Download do IntelliJ IDEA Community
Fonte: Elaborado pelo autor (2022)

Depois, navegue até o diretório onde o arquivo baixado está e execute-o para inicializar a instalação.

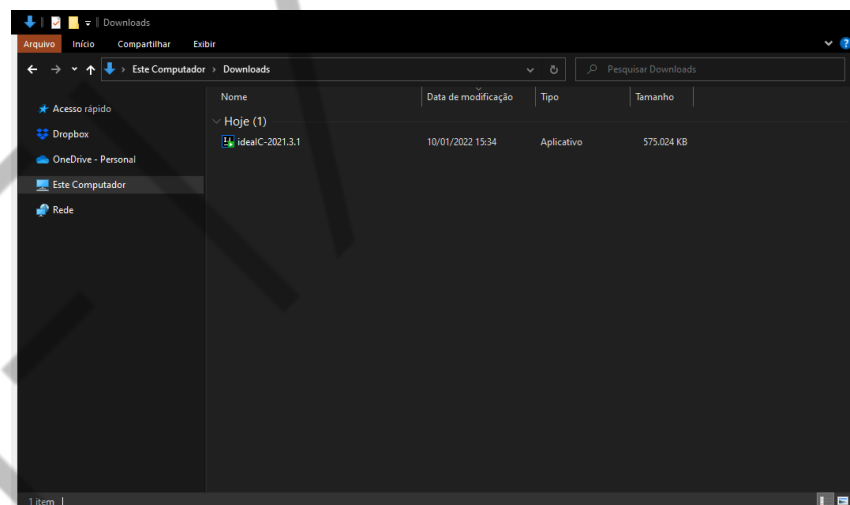


Figura 3 – Arquivo de instalação do IntelliJ IDEA
Fonte: Elaborado pelo autor (2022)

O processo de instalação será inicializado, clique em next.

Codando o primeiro código Java

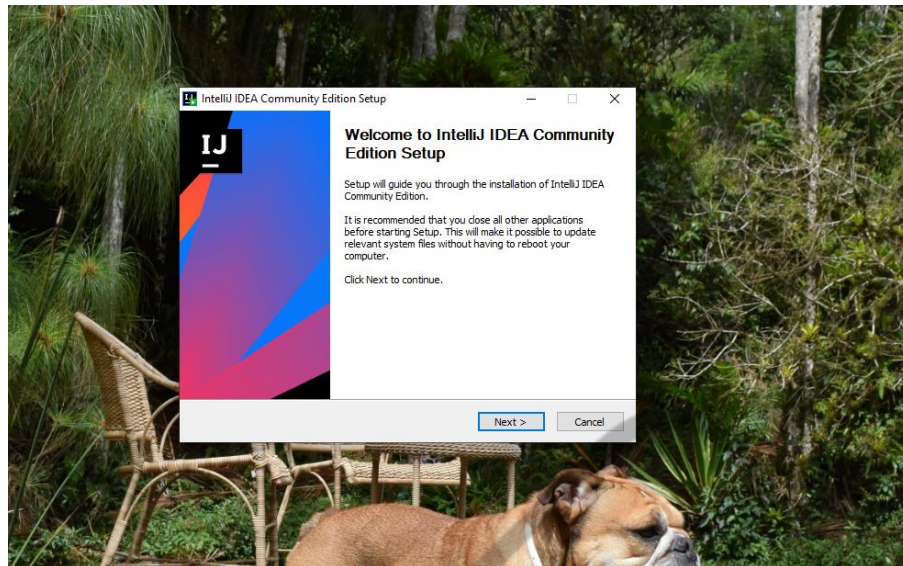


Figura 4 – Processo de instalação do IntelliJ IDEA (1)
Fonte: Elaborado pelo autor (2022)

Defina o diretório no qual será realizada a instalação, pode manter o diretório padrão, se preferir. Nesse passo também é possível verificar a quantidade de memória que será necessária (Figura “Processo de instalação do IntelliJ IDEA (2)”).

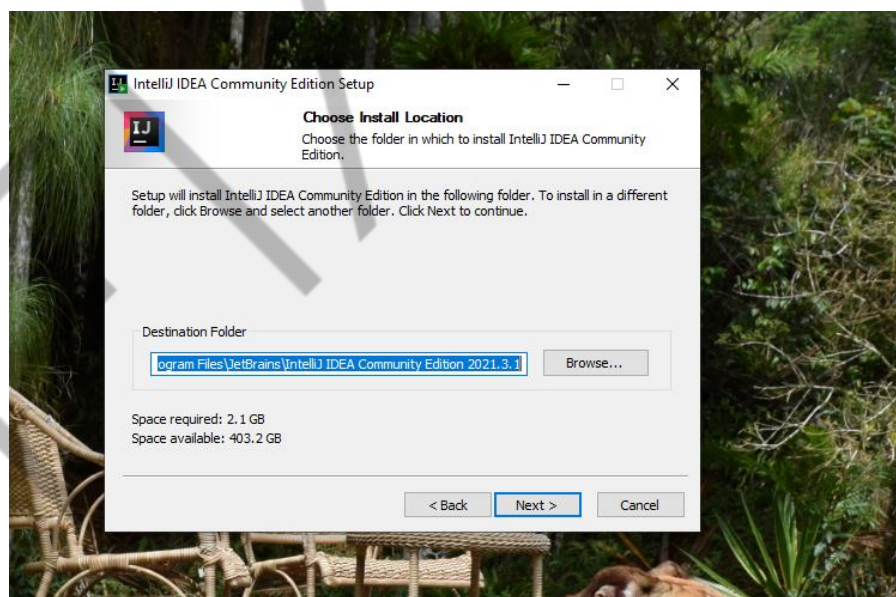


Figura 5 – Processo de instalação do IntelliJ IDEA (2)
Fonte: Elaborado pelo autor (2022)

Nesse passo, você pode configurar para criar um atalho no IntelliJ no Desktop, associar os arquivos com as extensões “.java”, “.groovy”, “.kt”, “.kts” para o IntelliJ. Não iremos marcar nenhuma opção, podemos somente prosseguir (Figura “Processo de instalação do IntelliJ IDEA (3)”).

Codando o primeiro código Java

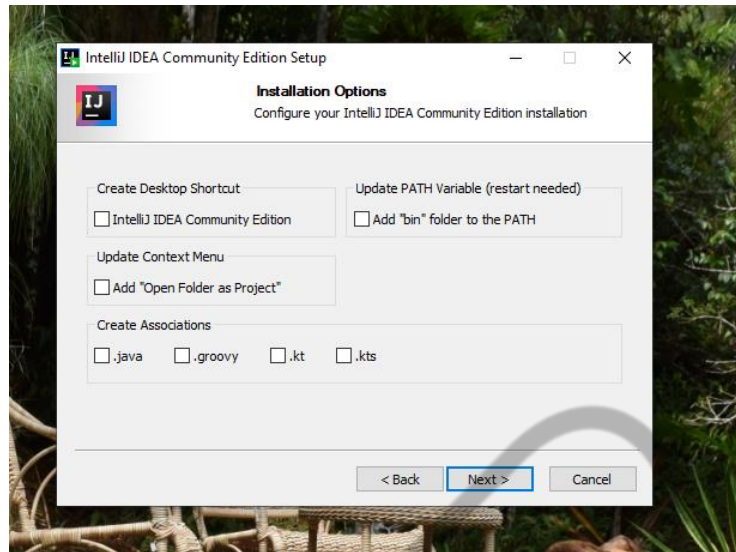


Figura 6 – Processo de instalação do IntelliJ IDEA (3)
Fonte: Elaborado pelo autor (2022)

O último passo antes da instalação é definir o local onde o atalho para inicialização do programa será criado. Pode deixar como está e clicar em “Install”. Resumindo, iniciamos o processo de instalação e deixamos tudo como estava (padrão) até realizar a instalação (Figura “Processo de instalação do IntelliJ IDEA (4)”).

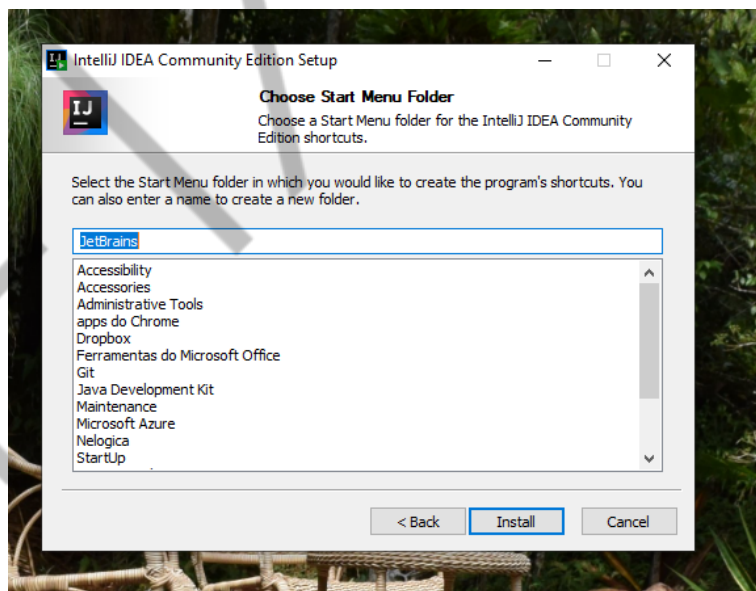


Figura 7 – Processo de instalação do IntelliJ IDEA (4)
Fonte: Elaborado pelo autor (2022)

Agora a instalação irá ocorrer (Figura “Processo de instalação do IntelliJ IDEA (5)”).

Codando o primeiro código Java

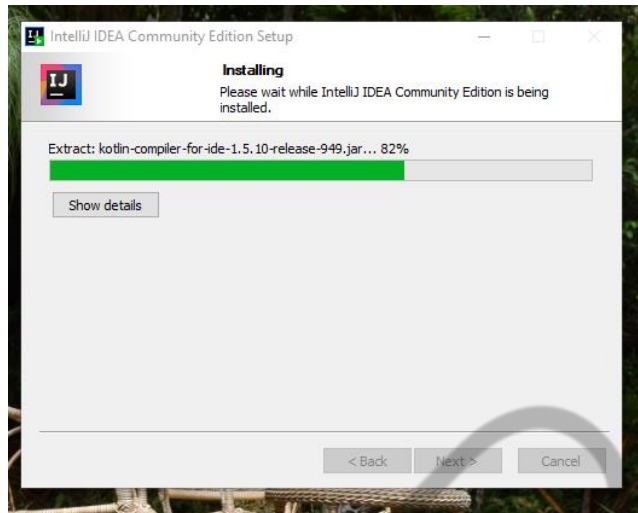


Figura 8 – Processo de instalação do IntelliJ IDEA (5)
Fonte: Elaborado pelo autor (2022)

Agora, sim! O processo de instalação foi finalizado! Podemos inicializar o IntelliJ (Figura “Processo de instalação do IntelliJ IDEA (6)”).

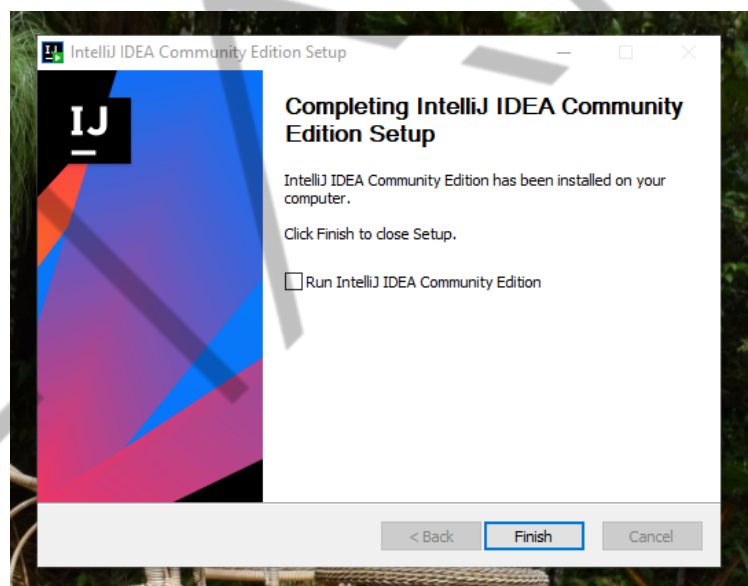


Figura 9 – Processo de instalação do IntelliJ IDEA (6)
Fonte: Elaborado pelo autor (2022)

Após inicializar o IntelliJ pela primeira vez, será necessário ler os termos de uso da ferramenta na versão community e estar de acordo. Marque o checkbox se estiver de acordo e clique em “continue”.

Codando o primeiro código Java

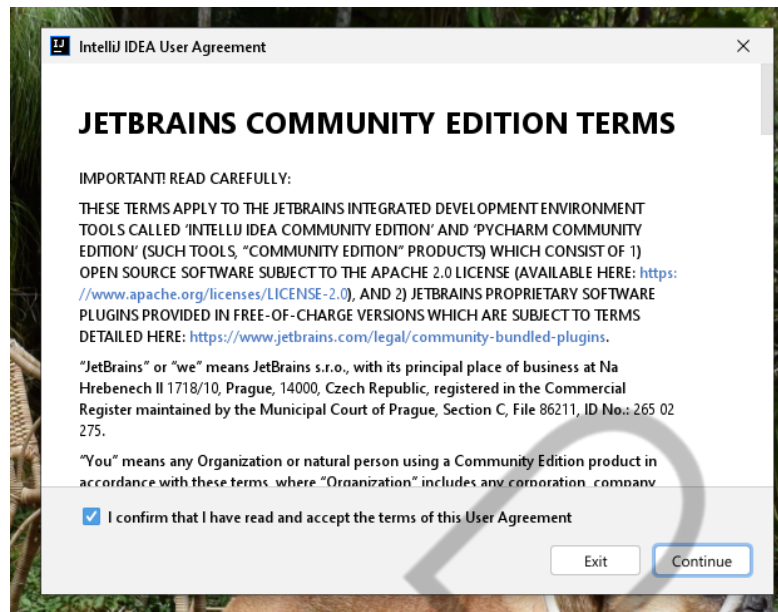


Figura 10 – Termos de uso do IntelliJ IDEA Community
Fonte: Elaborado pelo autor (2022)

Vamos criar o nosso primeiro projeto! Para isso, escolha a opção “New Project” (Figura “Criando um projeto Java no IntelliJ (1)”).

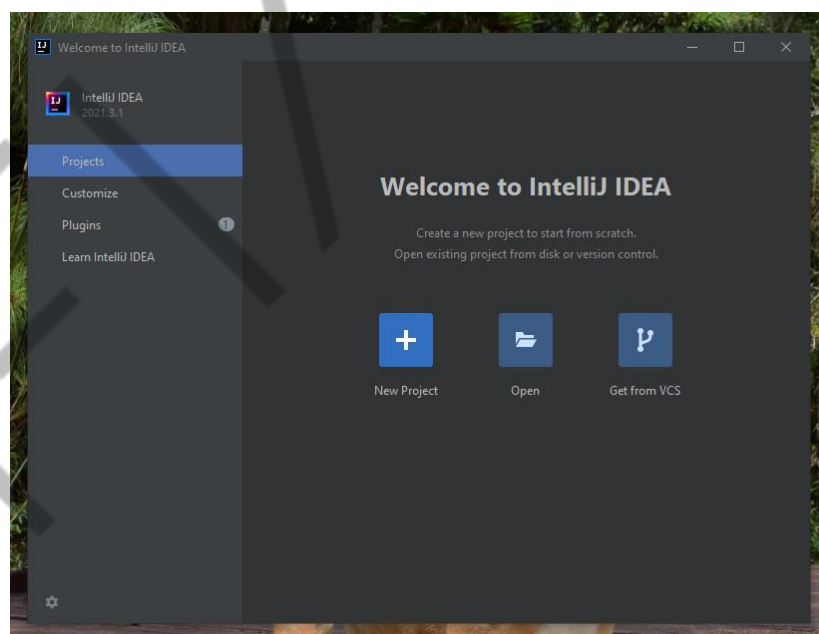


Figura 11 – Criando um projeto Java no IntelliJ (1)
Fonte: Elaborado pelo autor (2022)

Escolha a opção “Java” e clique em “Next” (Figura “Criando um projeto Java no IntelliJ (2)”).

Codando o primeiro código Java

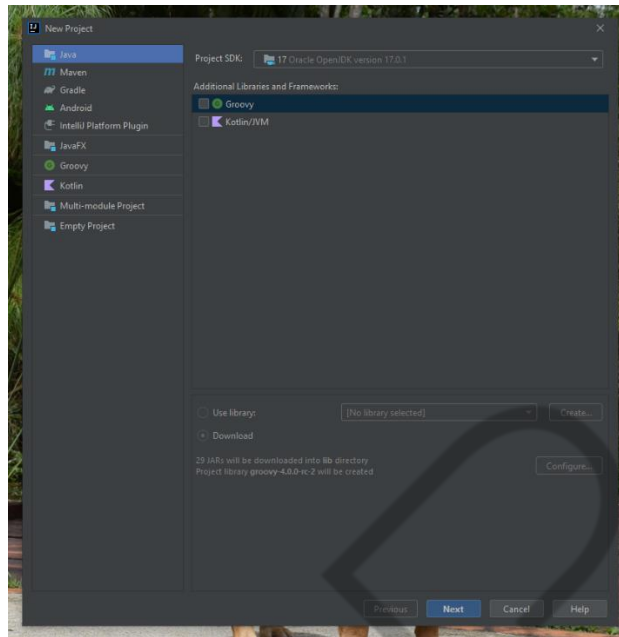


Figura 12 – Criando um projeto Java no IntelliJ (2)
Fonte: Elaborado pelo autor (2022)

Para criar um projeto com um template inicial, marque o checkbox da opção “Create Project from template” e escolha “Command Line App” (Figura “Criando um projeto Java no IntelliJ (3)”).

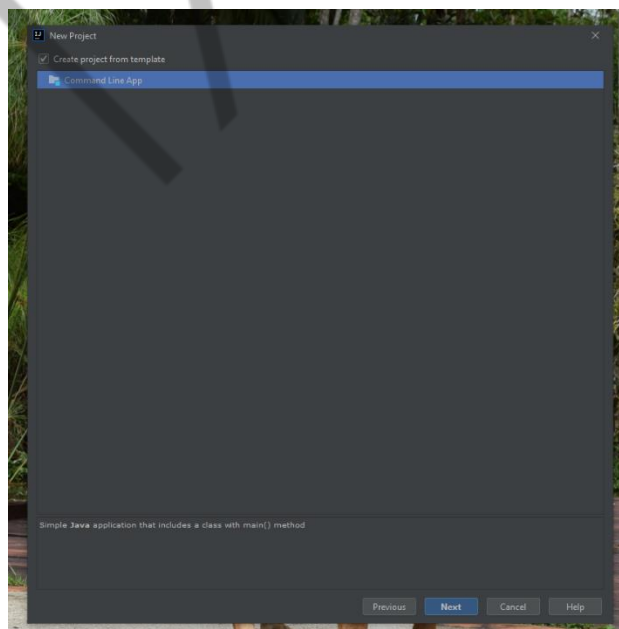


Figura 13 – Criando um projeto Java no IntelliJ (3)
Fonte: Elaborado pelo autor (2022)

Antes de finalizar a criação, precisamos definir o nome do projeto, o local onde serão salvos os arquivos e o nome do pacote.

Codando o primeiro código Java

O nome do projeto será “PrimeiroProjeto”, para o diretório você pode escolher um local de sua preferência ou deixar no diretório padrão. O nome do pacote será “br.com.fiap”, não se preocupe com essa nomenclatura ou com o que são os pacotes, veremos em detalhes logo mais!

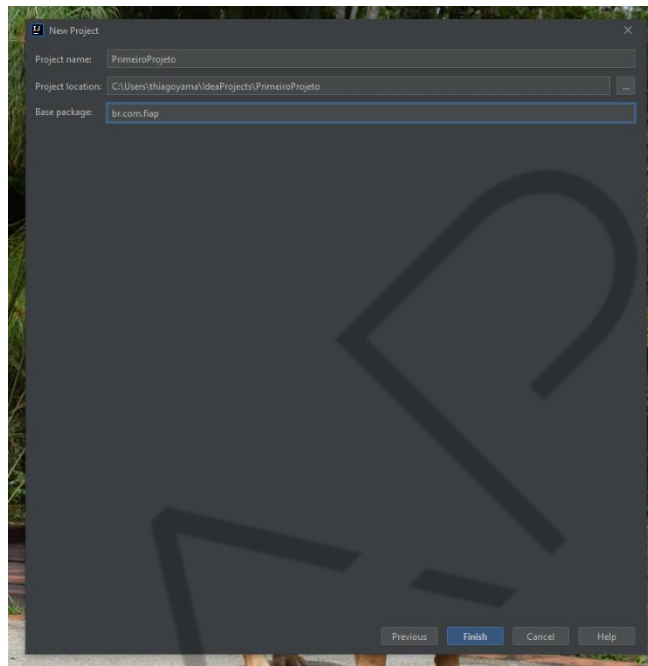


Figura 14 – Criando um projeto Java no IntelliJ (4)
Fonte: Elaborado pelo autor (2022)

Do lado esquerdo, temos a aba de “Project”, na qual podemos ver o projeto e seus arquivos. Do lado direito está o editor do código-fonte (Figura “Criando um projeto Java no IntelliJ (5)”).

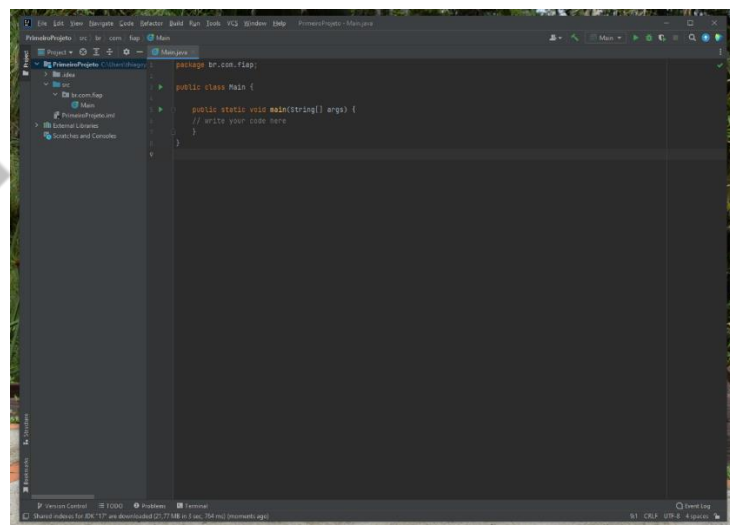


Figura 15 – Criando um projeto Java no IntelliJ (5)
Fonte: Elaborado pelo autor (2022)

Codando o primeiro código Java

Como criamos o projeto a partir de um template de App de linha de comando, a IDE criou de forma automática uma classe com o método main, o método principal do programa, pelo qual a execução do sistema irá começar.

Ajuste o código-fonte, adicionando o comando para escrever algum texto no console e depois execute o programa clicando no botão “play” no canto superior direito ou com o atalho “SHIFT + F10” (Figura “Execução do primeiro programa Java no IntelliJ”).

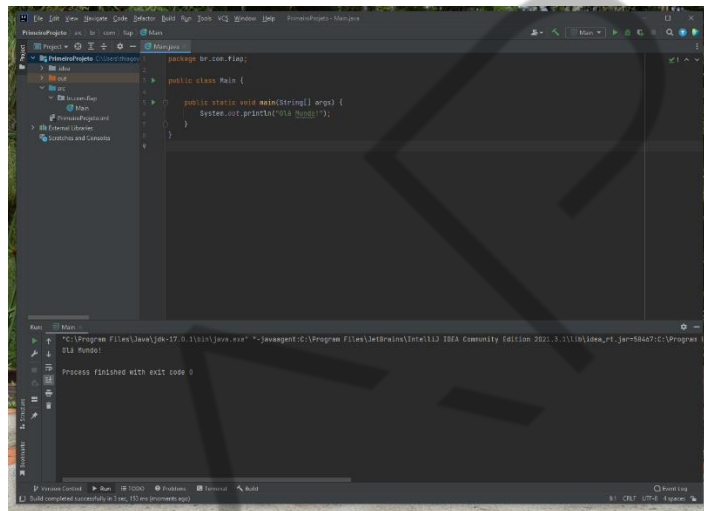


Figura 16 – Execução do primeiro programa Java no IntelliJ
Fonte: Elaborado pelo autor (2022)

Após executar, você poderá ver o resultado da execução na janela que será aberta. Observe também que um diretório chamado “out” foi criado na estrutura do projeto. Navegue até o diretório “fiap” e no arquivo “Main” clique com o botão direito do mouse e escolha a opção “Open in” > “Explorer” (Figura “Abrindo a classe Java compilado no Explorer”).

Codando o primeiro código Java

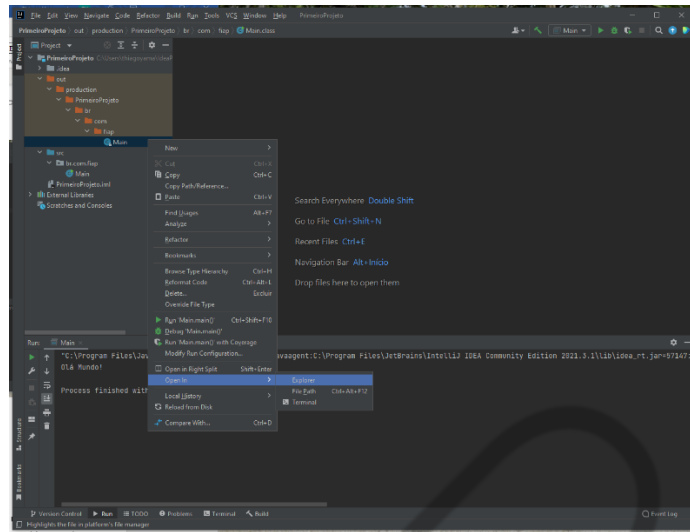


Figura 17 – Abrindo a classe Java compilado no Explorer
Fonte: Elaborado pelo autor (2022)

No primeiro capítulo, vimos como utilizar as ferramentas da JDK para compilar a classe e como interpretar o bytecode gerado na JVM por meio de linhas de comando. Observe que o mesmo aconteceu na IDE, porém de forma automática. O IntelliJ utilizou o compilador para gerar o bytecode e depois mandou interpretar na JVM para nós. Nesse diretório que abrimos é possível ver o código Java compilado (Figura “Verificando o bytecode gerado”).

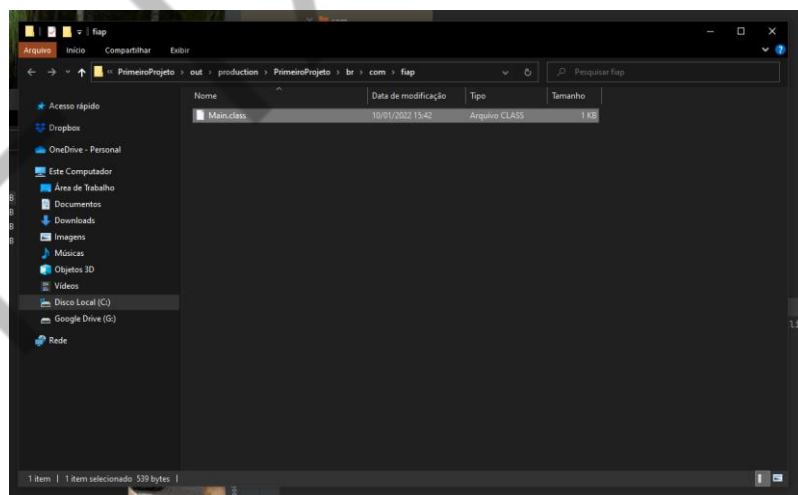


Figura 18 – Verificando o bytecode gerado
Fonte: Elaborado pelo autor (2022)

A pasta **src** é o local onde devemos criar os arquivos Java. Os arquivos java estão organizados em pacotes. Os **pacotes** são uma hierarquia de diretórios para a organização dos arquivos de uma aplicação Java.

Pense em um sistema grande e complexo como uma rede social ou em uma fintech que precisa trabalhar com milhares de informações de seus clientes e processar transações financeiras ou gerenciar milhares de posts e comentários.

Quantos arquivos de aplicações como essas devem ter? Muitos. Agora, imagine se todos os arquivos estivessem dentro do mesmo diretório, como seria a gestão de todos eles? Complicado, no mínimo. Por isso, precisamos dos pacotes, assim como utilizamos pastas (diretórios) para organizar os nossos arquivos pessoais, os pacotes irão organizar os arquivos da aplicação Java.

Os pacotes Java possuem uma hierarquia, assim como os diretórios e subdiretórios que criamos no computador. Além de organizar, os pacotes garantem a singularidade dos nomes das classes. Em um computador, não podemos ter dois arquivos com o mesmo nome no mesmo diretório, porém, podemos tê-los em diretórios diferentes.

E isso acontece com o Java, não é possível ter duas classes com o mesmo nome no mesmo pacote, porém, em pacotes diferentes é possível. Por exemplo, é possível ter duas classes Produto, desde que estejam em pacotes diferentes. Dessa forma, se o projeto utiliza uma biblioteca de classes fornecidas por outras empresas, não teremos conflito de nomes, já que cada classe terá o seu pacote correspondente.

No Java, assim para as outras linguagens, existem padrões de nomenclaturas, que são boas práticas utilizadas por todos os desenvolvedores para padronizar os nomes das classes, pacotes etc. Para os pacotes, o padrão é utilizar o nome de domínio da internet da empresa ao contrário. Por exemplo, para a universidade FIAP, que possui o domínio www.fiap.com.br, o pacote padrão deve ser a URL ao contrário, sem o “www”, dessa forma será **br.com.fiap**. E devemos especificar mais ainda os pacotes, com o nome do projeto, conjunto de classes etc. Assim, se o projeto for para o FIAP ON, o pacote deve ser **br.com.fiap.on**.

No IntelliJ foi definido no momento da criação do projeto o nome do pacote base (br.com.fiap). Podemos criar vários outros pacotes dentro do nosso projeto, para isso, clique com o botão direito do mouse na pasta **src** e escolha “New” > “Package” (Figura “Criando um pacote (1)”).

Codando o primeiro código Java

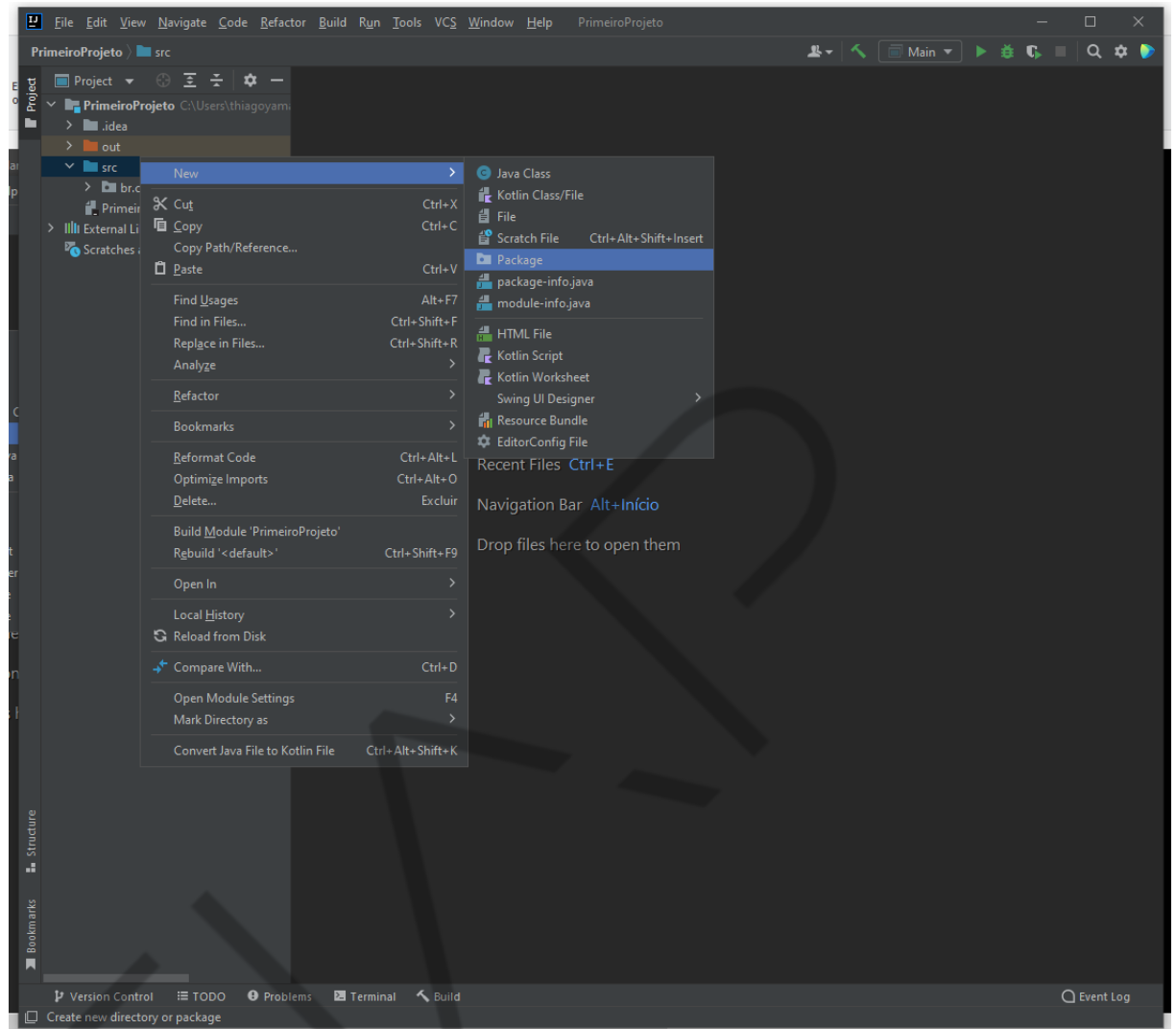


Figura 19 – Criando um pacote (1)
Fonte: Elaborado pelo autor (2022)

Agora, você deve dar o nome ao pacote e finalizar teclando Enter (Figura “Criando um pacote (2)”).

Codando o primeiro código Java

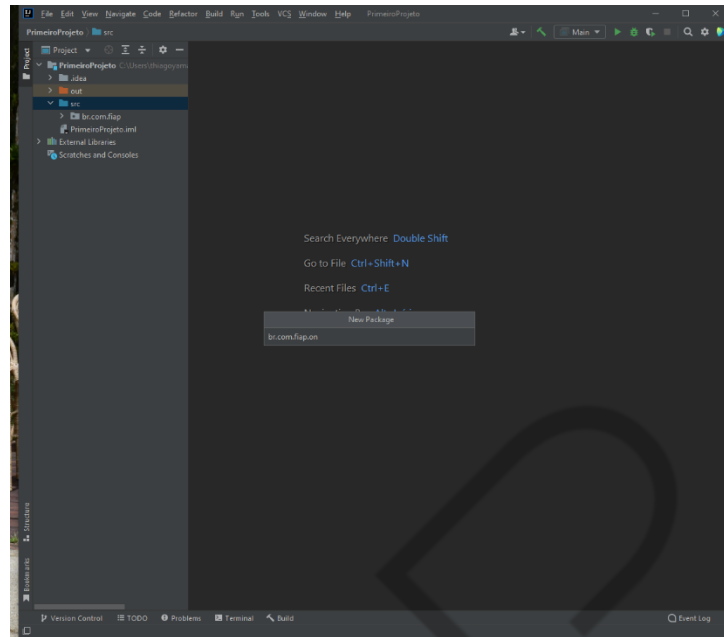


Figura 20 – Criando um pacote (2)
Fonte: Elaborado pelo autor (2022)

O resultado será um novo pacote dentro da estrutura **br.com.fiap** (Figura “Criando um pacote (3)”).

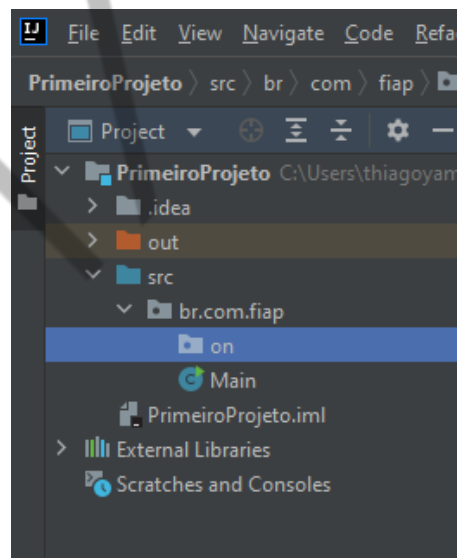
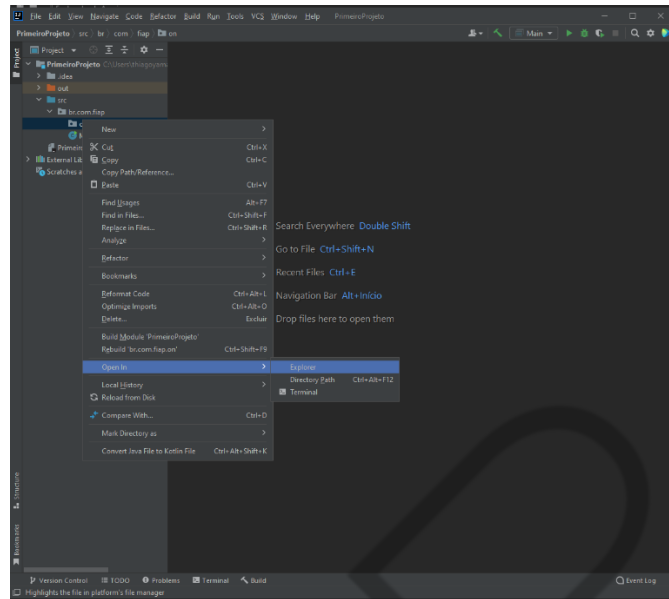


Figura 21 – Criando um pacote (3)
Fonte: Elaborado pelo autor (2022)

Navegue até a estrutura de diretórios que foi criada no disco rígido, você pode clicar com o botão direito do mouse no pacote/arquivo que deseja abrir no Explorer e escolher “Open in” > “Explorer” (Figura “Abrindo o pacote no Explorer do Windows”).



Fonte: Elaborado pelo autor (2022)

Codando o primeiro código Java

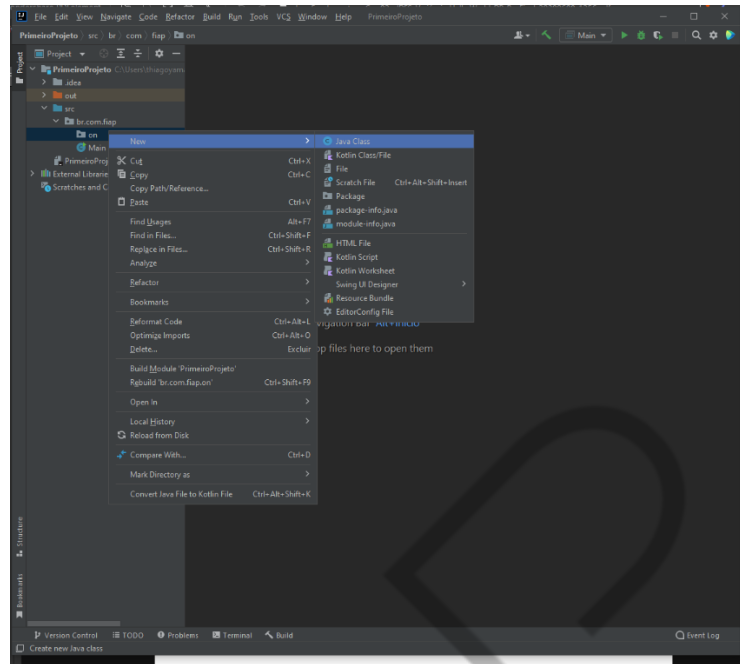


Figura 24 – Criando a classe Java (1)
Fonte: Elaborado pelo autor (2022)

Para finalizar, defina um nome para a classe e tecle Enter (Figura “Criando a classe Java (2)”).

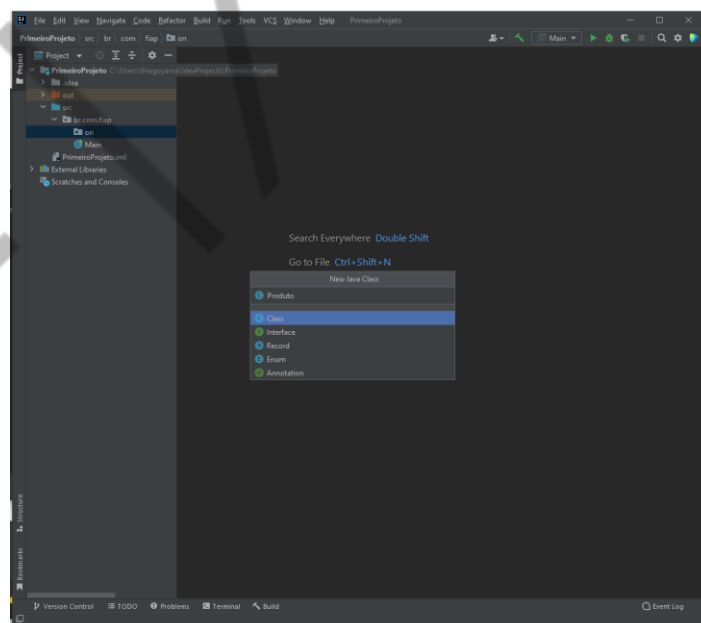


Figura 25 – Criando a classe Java (2)
Fonte: Elaborado pelo autor (2022)

Pronto! Nossa classe foi criada! Observe que a primeira linha da classe define o pacote da classe, por meio da palavra reservada **package**. Isso é uma regra, por isso, a classe possui na primeira linha a instrução: **package br.com.fiap.on.**

Codando o primeiro código Java

Outro ponto de atenção é para o padrão de nomenclatura das classes; para os pacotes, vimos que todos os caracteres são minúsculos. Para as classes, o padrão é conhecido como **UpperCamelCase**, ou seja, a primeira letra da classe é maiúscula e as demais minúsculas. Se o nome possuir mais de uma palavra, as outras palavras também devem começar com maiúscula, ex.: Pessoa, PessoaFisica e ContaCorrente.

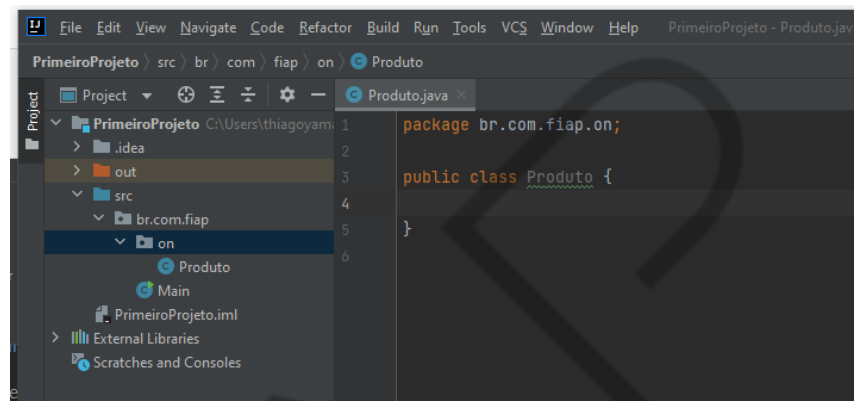


Figura 26 – Classe Produto
Fonte: Elaborado pelo autor (2022)

Depois da declaração do pacote temos a declaração da classe. A palavra reservada **class** define a classe, seguida do nome da classe. O modificador **public** define que a classe é pública, ou seja, todos podem acessá-la. Veremos em detalhes sobre os modificadores de acesso em breve. Uma regra importante é que o nome do arquivo deve ser o mesmo da classe Java, por isso, o arquivo criado no exemplo foi “Produto.java” e o nome da classe “Produto”, caso você altere algum desses nomes, o programa Java não irá mais compilar. Inclusive as letras maiúsculas e minúsculas são diferenciadas no Java, ou seja, fazem a diferença. Isso se chama ser **case sensitive**.

As chaves { } delimitam os blocos em um programa Java, neste caso, as chaves após o nome da classe estão delimitando o início e o fim da classe “Produto”.

Agora, vamos analisar a classe Main.

Codando o primeiro código Java

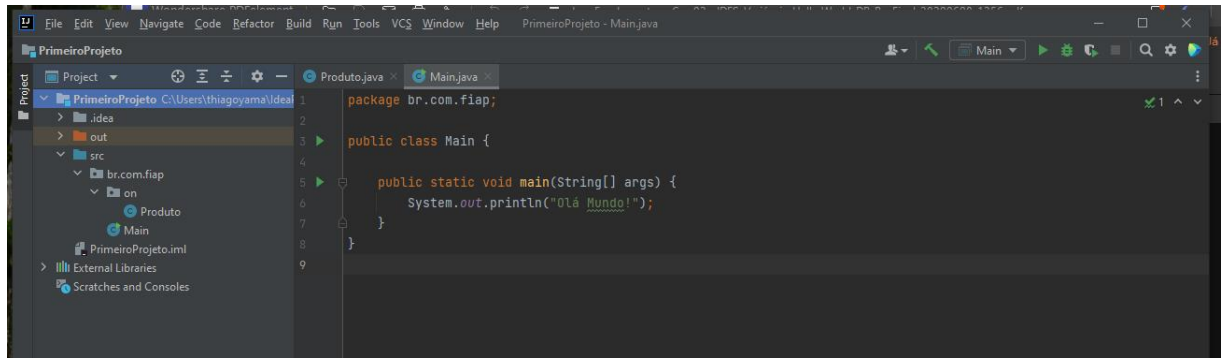


Figura 27 – Classe Main
Fonte: Elaborado pelo autor (2022)

Essa classe possui a definição de pacote e a declaração de classe de forma parecida com a classe **Produto**.

O que temos a mais nessa classe é o método **main**. (linha 5 do código, **public static void main(String[] args)**). As chaves estão delimitando o início e o fim do método, percebe que o método está dentro das chaves da classe **main** (Figura “Classe Main”).

Neste momento, não se preocupe com as palavras reservadas que estão definindo o método **main**, ou com o próprio método. Em breve iremos explorar de forma aprofundada tudo isso. O importante, neste momento, é entender que, para executar uma classe compilada, a máquina virtual Java (JVM) sempre inicia a execução do programa pelo método **main**. Portanto, é necessário ter um método **main** na classe que você deseja executar.

Podemos criar várias classes e chamá-las a partir do método **main**, ou seja, não é necessário que todas as classes tenham esse método.

A instrução que adicionamos anteriormente, **System.out.println(“Olá Mundo”);**, que foi adicionada dentro do método **main**, exibe no console a informação que está dentro dos parênteses, neste caso, **Olá Mundo**. Toda instrução deve terminar com o ponto e vírgula “;”.

2 TIPOS DE DADOS

Para trabalhar com informações dentro de um programa, precisamos de variáveis. Por exemplo, para armazenar o nome de um cliente, a data de vencimento

Codando o primeiro código Java

ou o valor de um produto, vamos declarar variáveis no programa Java definindo o tipo de informação que será armazenado e o seu nome. A sintaxe básica é:

```
<tipo da variável> <nome da variável>;
```

Exemplos:

```
int idade;
```

```
double preco;
```

É possível declarar mais de uma variável do mesmo tipo na mesma linha:

```
float altura, peso;
```

A linguagem Java é fortemente tipada, pois cada variável deve definir o tipo de dado que irá armazenar. Além disso, temos dois tipos principais: tipos primitivos e tipos de referência. Existem oito tipos primitivos dentro da linguagem Java, um tipo primitivo não é um objeto e sim uma parte interna da linguagem Java que armazena somente a informação, tornando-os mais eficientes. O outro tipo, de referência, armazena um objeto. Esse tipo veremos com detalhes mais adiante.

Um ponto importante são as regras de nomenclatura, já vimos os padrões para nome de pacotes e classes. Para as variáveis o padrão é o **camelCase**, ou seja, a primeira letra da variável é minúscula e as demais palavras começam com a letra maiúscula. Os nomes das variáveis podem começar com uma letra, um caractere de sublinhado (_) ou \$. Depois do primeiro caractere, os nomes das variáveis podem conter quaisquer combinações de letras ou números. Geralmente não utilizamos acentos ou outros caracteres especiais para definir o nome de variáveis, pacotes, classes etc.

Lembre-se de que a linguagem Java é case sensitiva, dessa forma, uma variável **preco** é diferente de **Preco**. Observe também que não utilizamos cedilha no nome da variável.

Existem também palavras reservadas dentro da linguagem que não podem ser utilizadas para nomear as variáveis, classes ou métodos, pois possuem um significado dentro da programação. A lista completa está abaixo, mas não precisa se preocupar em decorar, pois, caso você utilize alguns desses nomes, a IDE irá avisar e o programa não irá compilar (Tabela “Palavras reservadas”).

Tabela 1 – Palavras reservadas (1)

abstract	continue	For	New	switch
assert	default	Goto	Package	synchronized
boolean	do	If	Private	This
break	Double	implements	protected	Throw
byte	Else	Import	Public	Throws
case	Enum	instanceof	Return	Transiente
catch	extends	Int	Short	Try
char	Final	Interface	static	Void
class	Finally	Long	strictfp	Volatile
const	Float	Native	super	While

Fonte: Elaborado pelo autor (2022)

Voltando aos tipos de variáveis, como já adiantamos, existem oito tipos primitivos. Quatro para armazenar **números inteiros**, sem a parte fracionária. Dois para **números reais**, com a parte fracionária. Um para armazenar um **único caractere**, como letras, algarismos, sinais de pontuação, entre outros. O último tipo primitivo é o **lógico (boolean)**, que possui somente dois valores: verdadeiro (true) ou falso (false). No Java não é possível converter números inteiros em valores booleanos.

Tabela 2 – Palavras reservadas (2)

Tipo	Tamanho em bits	Valores (min. e max.)
Boolean	8	true ou false
Char	16	0 a 65.535
Byte	8	-128 a +127

Codando o primeiro código Java

Short	16	-32.768 a +32.767
Int	32	-2.147.483.648 a +2.147.483.647
Long	64	-9.223.372.036.854.775.808 a +9.223.372.036.854.775.807
Float	32	1,40129846432481707E-45 a 3,40282346638528860E+38
Double	64	4,94065645841246544E-324 a 1,79769313486231570E+308

Fonte: Elaborado pelo autor (2022)

Para os números inteiros, temos o **byte**, **short**, **int** e **long**. A diferença entre eles é o tamanho do número que consegue armazenar e, consequentemente, a quantidade de memória necessária para isso. Assim, por exemplo, se você precisa armazenar a idade de uma pessoa, o tipo **short** é o suficiente, já que o número máximo positivo que ele pode armazenar é 32.767, já para o **id** dos **clientes**, o tipo **int** ou **long** serão melhores.

Valores de **ponto flutuante** são os números que possuem a parte fracionária, ou seja, a parte decimal. Os dois tipos são **float** e **double**, a diferença entre eles também é o tamanho, o tipo **double** é duas vezes mais preciso que o tipo **float**. Assim, se precisamos armazenar a nota de um aluno ou o peso, o tipo **float** será suficiente, já para armazenar o faturamento de uma grande empresa, o tipo **double** é o mais indicado.

O **char** armazena um único caractere, para armazenar mais caracteres, como uma palavra ou texto, precisamos de outro tipo de dado, a **String** que veremos logo mais.

Codando o primeiro código Java

2.1 Manipulando dados

Para associar um valor a uma variável, utilizamos o operador de atribuição, que é definido pelo sinal de igual (=).

Exemplos:

`int idade = 10;`

`float peso = 80.5;`

`char sexo = 'M';`

`boolean especial = false;`

Observe que, para associar a parte fracionária de um número, utilizamos o ponto (.) e não a vírgula (,). Outro detalhe é o char, o valor deve ser definido utilizando aspas simples ('). Já para o boolean, os valores aceitos são true ou false, sem aspas.

Agora é sua vez! Crie uma classe com o método main, declare algumas variáveis, atribua valores e imprima o seu conteúdo (Figura “Exemplo de tipos primitivos”).

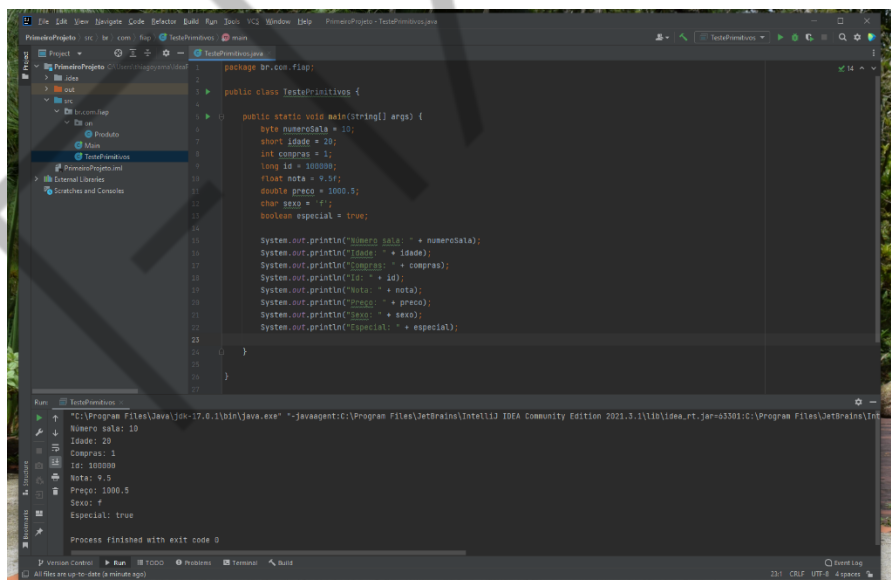


Figura 28 – Exemplo de tipos primitivos

Fonte: Elaborado pelo autor (2022)

Perceba que para o tipo **float** foi necessário adicionar um “f” no final do número para definir que o valor atribuído é do tipo **float**, pois o Java entende que o número

Codando o primeiro código Java

com ponto “.” é do tipo **double**. E como o **double** pode armazenar um número maior do que um **float** é preciso deixar explícito que queremos o valor do tipo correto.

Na hora de imprimir o valor, utilizamos o “+” para concatenar o valor do texto que está entre as aspas (“”) e o valor que está dentro da variável, para ficar mais legível na hora de ver as informações impressas.

Se você criou a classe no mesmo projeto, na hora de executar você deve clicar com o botão direito do mouse na classe e escolher “Run NomeDaClasse.main()”, pois, como o projeto tem mais de uma classe com o método main, você deve dizer qual classe deseja executar.

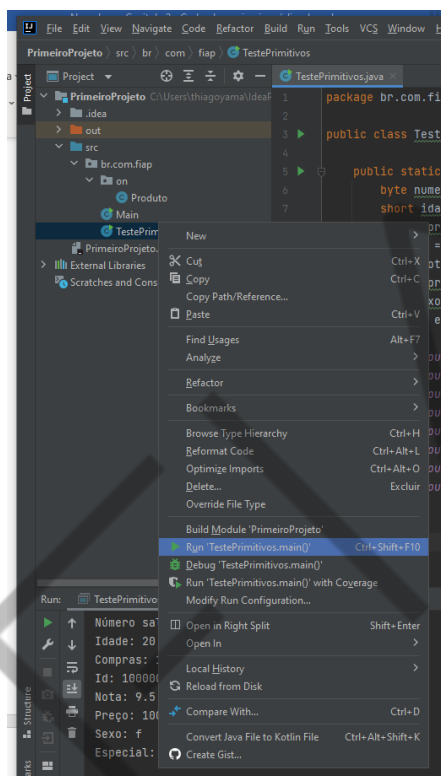


Figura 29 – Executando a classe
Fonte: Elaborado pelo autor (2022)

2.1.1 Operadores aritméticos

Para realizar operações matemáticas, atribuições e comparações lógicas utilizamos símbolos especiais chamados de operadores. São cinco símbolos que realizam as operações matemáticas, os operadores aritméticos.

Codando o primeiro código Java

Os operadores aritméticos são: soma (+), subtração (-), multiplicação (*), divisão (/) e o módulo (%), que retorna o resto da divisão de dois números inteiros.

A divisão entre dois números inteiros resulta em um número inteiro, sem a parte fracionária, pois é descartada e não faz nenhum arredondamento. Por exemplo, o resultado de $5/2$ é 2 e não 2.5. O operador módulo fornece o resto da divisão, dessa forma, $5 \% 2$ produz o resultado 1.

Assim como na matemática, no Java é possível realizar várias operações aritméticas de uma vez. As regras de precedência de operações aplicadas na matemática são as mesmas no Java. As operações de multiplicação (*), divisão (/) e módulo (%) possuem prioridade em relação às operações de soma (+) e subtração (-). Por exemplo: o resultado da expressão $5 * 2 + 1$ é 11, pois primeiro é realizada a multiplicação ($5 * 2$) e depois a soma ($10 + 1$).

Utilizamos os parênteses para agrupar as operações matemáticas, da mesma forma que são utilizados em expressões algébricas. Assim, a expressão $5 * (2 + 1)$ produz o resultado 15. Primeiro é realizada a soma de $2 + 1$ e depois a multiplicação entre o resultado da soma, 3 e 5, o que resulta em 15.

Observe os operadores aritméticos (Tabela “Operadores aritméticos”).

Tabela 3 – Operadores aritméticos

Operação	Operador	Exemplo
Soma	+	$4 + 4$
Subtração	-	$4 - 4$
Multiplicação	*	$4 * 4$
Divisão	/	$4 / 4$
Módulo	%	$4 \% 4$

Fonte: Elaborado pelo autor (2022)

2.1.2 Operadores de atribuição

Para atribuir um valor a uma variável, existe um operador de atribuição, como já vimos. A atribuição ocorre sempre após o processamento das expressões do lado direito da atribuição. Por exemplo:

```
int x = 2;
```

```
x = x + 10;
```

Qual o valor final de x? Primeiro, a variável x recebeu o valor 2, depois, a expressão $x + 10$ é processada, resultando no valor 12, que por fim é atribuído novamente à variável x.

Existem também os operadores de atribuição que realizam a operação aritmética e atribuem o resultado da operação à própria variável. Existem essas operações para cada operação aritmética.

Exemplo:

```
int x = 2;
```

```
x+=10; // x = x + 10;
```

Apresentamos os operadores de atribuição e as expressões às quais são equivalentes (Tabela “Operadores de atribuição”).

Tabela 4 – Operadores de atribuição

Operador	Expressão	Equivalente
+=	$x += y;$	$x = x + y;$
-=	$x -= y;$	$x = x - y;$
*=	$x *= y;$	$x = x * y;$
/=	$x /= y;$	$x = x / y;$
%=	$x \% = y;$	$x = x \% y;$

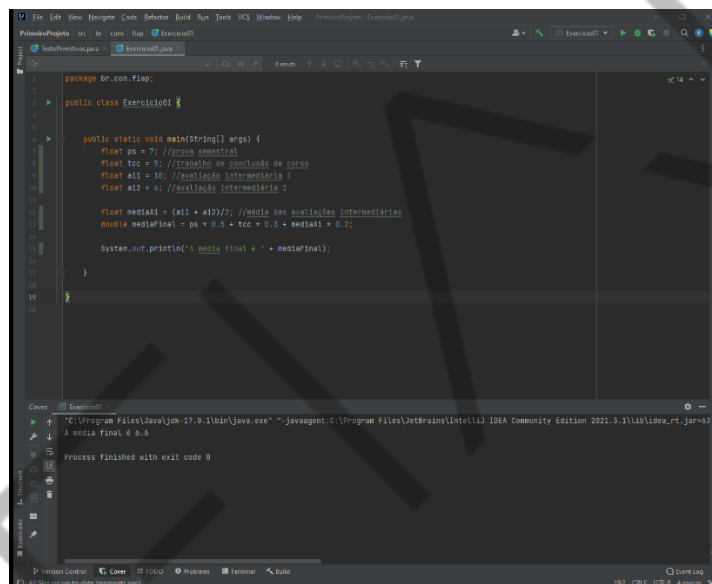
Fonte: Elaborado pelo autor (2022)

Codando o primeiro código Java

Agora é a sua vez! Vamos calcular a média de um estudante de uma faculdade. A média final é composta pela prova semestral, que compõe 50% da média, trabalho de conclusão de curso, que responde por 30%, e as avaliações intermediárias, que são compostas por duas avaliações durante o semestre e a média dessas duas avaliações contribuem com 20% para a média final.

Vamos supor que o aluno tirou 7 na prova semestral, 5 no trabalho de conclusão, 10 e 6 nas provas intermediárias. Qual será a sua média final do semestre?

Declare as variáveis e atribua os valores correspondentes. Depois, declare uma variável para calcular a média das avaliações intermediárias e realize o cálculo. Por fim, declare uma variável para armazenar a média final, realize o cálculo e exiba o resultado.



```
package br.com.flap;

public class Exercício01 {

    public static void main(String[] args) {
        float ps = 7; //prova semestral
        float tcc = 5; //trabalho de conclusão de curso
        float a11 = 10; //avaliação intermediária 1
        float a12 = 6; //avaliação intermediária 2

        float media1 = (a11 + a12) / 2; //média das avaliações intermediárias
        double mediaFinal = ps * 0.5 + tcc * 0.3 + media1 * 0.2;

        System.out.println("A média final é " + mediaFinal);
    }
}
```

Output: A média final é 6.8

Figura 30 – Exercício 01
Fonte: Elaborado pelo autor (2022)

2.1.3 Operadores de incremento e decremento

Existem também os operadores de decremento e incremento que aumentam ou diminuem o valor de uma variável em uma unidade.

Por exemplo, a variável `x` possui o valor 10; para somar 1 à variável, basta escrever `x++`, assim, o valor final será 11. É o mesmo resultado da expressão `x = x + 1`; . Para subtrair 1 de `x`, basta escrever `x--`, que produz o mesmo resultado de `x = x - 1`;

Codando o primeiro código Java

Sabendo disso, quais os valores impressos na execução do programa abaixo (Figura “Operadores de incremento e decremento”)?

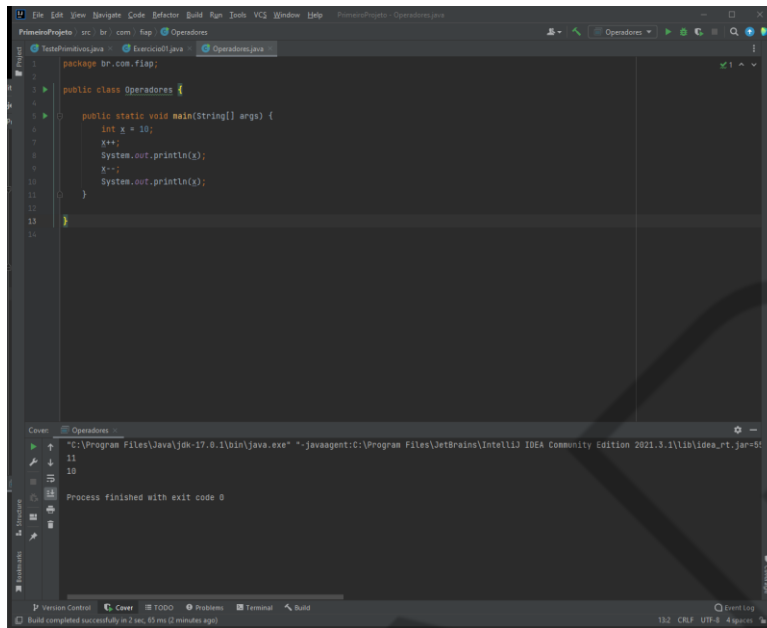
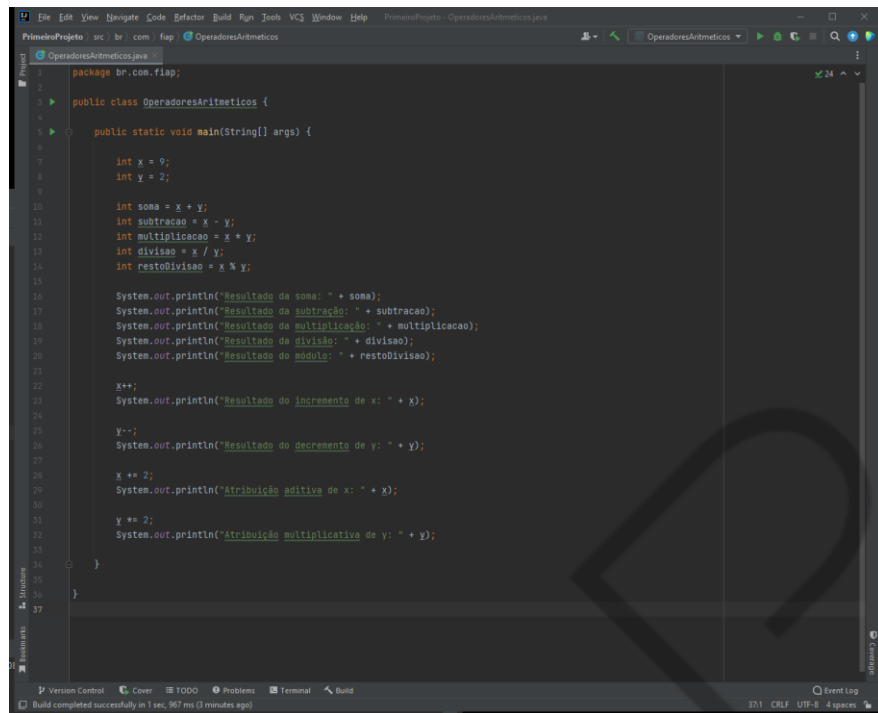


Figura 31 – Operadores de incremento e decremento
Fonte: Elaborado pelo autor (2022)

Como podemos ver, a resposta é 11 e 10, pois primeiro x recebe 10 e depois utilizamos o operador de incremento para adicionar 1 à variável x. Depois, com x = 11, decrementamos em 1 e a imprimimos, assim x retorna ao valor original, 10.

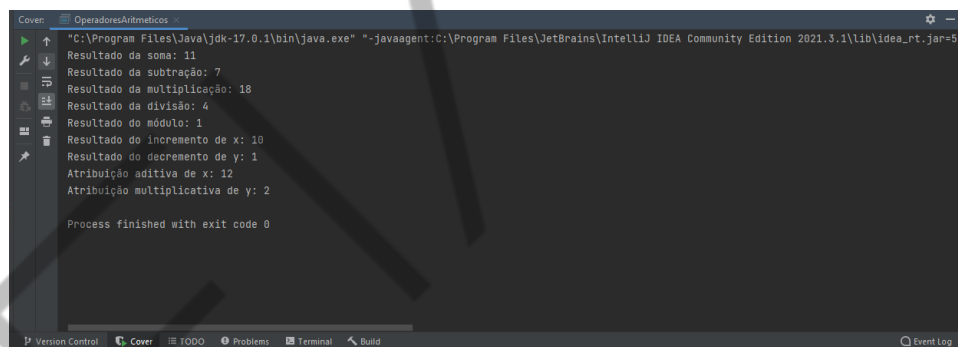
Agora, para praticar, vamos criar uma classe e utilizar todos os operadores vistos até o momento (Figuras “Exemplo da utilização dos operadores” e “Resultado da execução”).

Codando o primeiro código Java



```
1 package br.com.flap;
2
3 public class OperadoresAritmeticos {
4
5     public static void main(String[] args) {
6
7         int x = 9;
8         int y = 2;
9
10        int soma = x + y;
11        int subtracao = x - y;
12        int multiplicacao = x * y;
13        int divisao = x / y;
14        int restoDivisao = x % y;
15
16        System.out.println("Resultado da soma: " + soma);
17        System.out.println("Resultado da subtração: " + subtracao);
18        System.out.println("Resultado da multiplicação: " + multiplicacao);
19        System.out.println("Resultado da divisão: " + divisao);
20        System.out.println("Resultado do módulo: " + restoDivisao);
21
22        x++;
23        System.out.println("Resultado do incremento de x: " + x);
24
25        y--;
26        System.out.println("Resultado do decremento de y: " + y);
27
28        x += 2;
29        System.out.println("Atribuição aditiva de x: " + x);
30
31        y *= 2;
32        System.out.println("Atribuição multiplicativa de y: " + y);
33    }
34}
```

Figura 32 – Exemplo da utilização dos operadores
Fonte: Elaborado pelo autor (2022)



```
Resultado da soma: 11
Resultado da subtração: 7
Resultado da multiplicação: 18
Resultado da divisão: 4
Resultado do módulo: 1
Resultado do incremento de x: 10
Resultado do decremento de y: 1
Atribuição aditiva de x: 12
Atribuição multiplicativa de y: 2

Process finished with exit code 0
```

Figura 33 – Resultado da execução
Fonte: Elaborado pelo autor (2022)

REFERÊNCIAS

BARNES, David J. **Programação Orientada a Objetos com Java: Uma introdução Prática Utilizando Blue J.** São Paulo: Pearson, 2004.

COELHO, Alex. **Java com Orientação a Objetos.** Rio de Janeiro: Ciência Moderna, 2012.

DEITEL P, DEITEL H. **Java Como Programar.** 10. ed. São Paulo: Pearson, 2016.

HORSTMANN, Cay; CORNELL, Gary. **Core Java. Volume I - Fundamentos.** 8.ed. São Paulo: Pearson, 2009.

SIERRA, Kathy; BATES, Bert. **Use a cabeça! Java.** Rio de Janeiro: Alta Books, 2010.